NeuronSeqSampler: A Hybrid Neural-Rhythmic Audio Sequencer

Jaakko Prättälä

January 3, 2026

Abstract

We present NeuronSeqSampler, a novel real-time audio sequencing system that integrates artificial neural network principles with advanced rhythm interpretation for generative music creation. The system employs a user-configurable spiking neural network coupled with a multi-band rhythmogram analyzer implementing Todd's (1994) auditory primal sketch model for rhythmic grouping. The architecture features: (1) an 8-band logarithmic filter bank for multi-scale temporal decomposition covering 0.125Hz-16Hz, (2) a recurrent neural network with per-neuron configurable activation functions (Linear, Sigmoid, ReLU, Tanh), (3) a BeatTracker-inspired multi-agent beat tracking system with competing tempo/phase hypotheses, (4) a real-time musical quantization engine with grid-based trigger suppression, (5) an adaptive rhythmogram-to-neuron connection matrix with dual-gain architecture, and (6) a comprehensive JSON-based preset management system. The system achieves approximately 13ms total latency while processing audio at 44.1kHz with 512-sample buffers, making it suitable for live performance and interactive composition. Drawing on principles from Todd's rhythmic analysis, Laine's neural modeling approach for musical notes, and modern multi-agent beat tracking research, our implementation enables emergent musical behaviors through the integration of biological neural dynamics, musical timing structures, and real-time performance control.

1 Introduction

Generative music systems have traditionally operated either through rule-based algorithmic composition or through neural network approaches trained on large datasets. NeuronSeqSampler bridges these paradigms by implementing a real-time system where a configurable neural network responds to rhythmic features extracted from audio input, creating an interactive performance tool that learns and adapts to musical context.

The system architecture consists of three primary components:

1. A rhythmogram analyzer implementing multi-scale temporal decomposition [2]
2. A recurrent spiking neural network with configurable topology
3. An adaptive learning system with onset-weighted updates

2 Rhythmogram Analysis

2.1 Theoretical Foundation

Following Todd's (1994) auditory primal sketch model [2], we implement a multi-scale rhythmic analysis that decomposes the input signal into musically relevant frequency bands. Unlike traditional spectral analysis, the rhythmogram focuses on temporal periodicities corresponding to musical rhythms.

2.2 Filter Bank Architecture

The system employs a logarithmically-spaced bank of bandpass filters covering the range 0.125 Hz to 16 Hz, corresponding to musical tempos from approximately 7.5 to 960 BPM:

$$f_i = f_{\min} \cdot \left(\frac{f_{\max}}{f_{\min}} \right)^{i/(N-1)}, \quad i = 0, 1, \ldots, N-1 \tag{1}$$

where N is the number of bands (default 8), $f_{\min} = 0.125$ Hz, and $f_{\max} = 16$ Hz.

Each band i is implemented as a second-order IIR bandpass filter with transfer function:

$$H_i(z) = \frac{b_0 + b_1 z^{-1} + b_2 z^{-2}}{1 + a_1 z^{-1} + a_2 z^{-2}} \tag{2}$$

The bandwidth of each filter is set proportionally to its center frequency:

$$BW_i = k \cdot f_i \tag{3}$$

where k is a scaling factor (default 0.5), ensuring constant-Q behavior.

2.3 Onset Detection

For each frequency band, we implement onset detection. The onset detection function computes the spectral flux in each band:

$$\Phi_i(n) = \max(0, |X_i(n)| - |X_i(n-1)|) \tag{4}$$

where $X_i(n)$ is the filter output for band i at time n. An onset is detected when:

$$\Phi_i(n) > \theta \cdot \mu_i + \epsilon \tag{5}$$

where θ is a threshold parameter, μ_i is a local mean of the onset function, and ϵ is a small constant to prevent spurious detections during silence.

2.4 Signal Flow Diagram — Rhythmogram

The rhythmogram processing pipeline follows this architecture:

```
Audio Input
    ↓

  Amplitude Envelope Extraction
  (RMS with 512-sample window)

    ↓

  Bandpass Filter Bank (8 bands)
  f = 0.125 Hz → f = 16 Hz
  (logarithmic spacing)

    ↓

  Band 0       Band 1       ...      Band 7
  (0.125 Hz)   (0.25 Hz)             (16 Hz)

    ↓            ↓            ↓
```

```
Onset         Onset         Onset
Detector      Detector      Detector

   ↓             ↓             ↓

 Filter Outputs: [f, f, ..., f]
 Onset Events: {(t, s, i)}
 where t=time, s=strength, i=band
```

3 Neural Network Architecture

3.1 Neuron Model

Each neuron j in the network is modeled as a leaky integrate-and-fire unit following Laine's (2000) [1] neural approach to musical note modeling, with activation function ϕ_j:

$$a_j(t+1) = \phi_j \left(\lambda_j \cdot a_j(t) + \sum_{k \in \mathcal{P}(j)} w_{kj} \cdot s_k(t) + r_j(t) \right) \tag{6}$$

where:

- $a_j(t)$ is the activation of neuron j at time t
- $\lambda_j \in [0, 1]$ is the leak rate (decay factor)
- $\mathcal{P}(j)$ is the set of neurons with connections to j
- w_{kj} is the weight from neuron k to neuron j
- $s_k(t)$ is the spike output of neuron k (1 if fired, 0 otherwise)
- $r_j(t)$ is the rhythmic input to neuron j

The activation function ϕ_j can be selected from:

- Linear: $\phi(x) = x$
- Sigmoid: $\phi(x) = \frac{1}{1+e^{-x}}$
- ReLU: $\phi(x) = \max(0, x)$
- Tanh: $\phi(x) = \tanh(x)$

3.2 Firing Threshold

A neuron fires when its activation exceeds its threshold θ_j:

$$s_j(t) = \begin{cases} 1 & \text{if } a_j(t) \geq \theta_j \\ 0 & \text{otherwise} \end{cases} \tag{7}$$

Upon firing, the neuron triggers its associated audio sample and its activation is reset:

$$a_j(t) \leftarrow 0 \tag{8}$$

3.3 Rhythmic Input Mapping

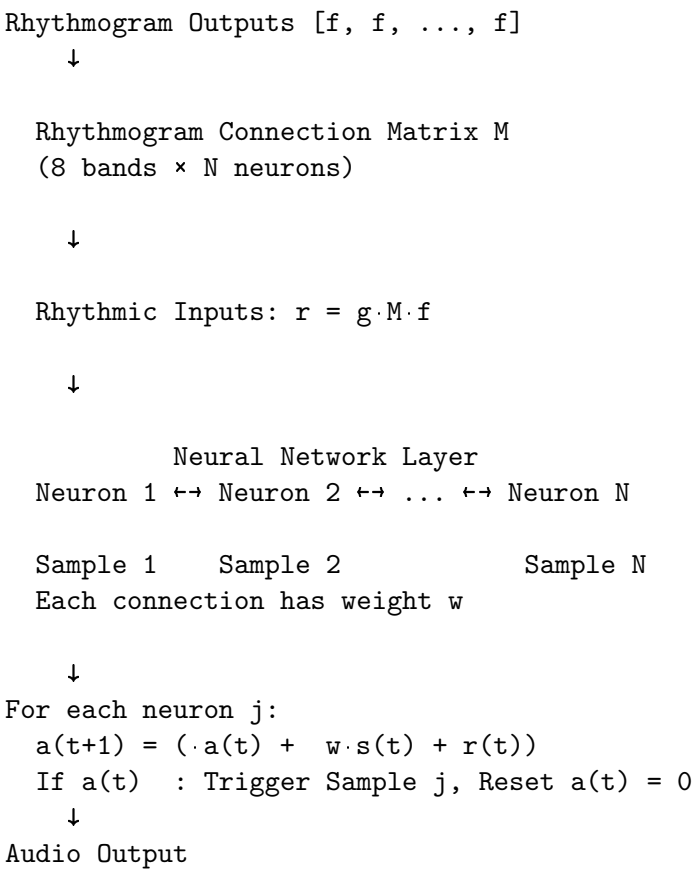
The rhythmic input $r_j(t)$ for neuron j is computed from the rhythmogram outputs via a learned connection matrix M:

$$r_j(t) = g \cdot \sum_{i=0}^{N-1} M_{ij} \cdot f_i(t) \tag{9}$$

where:

- M_{ij} is the connection weight from rhythm band i to neuron j
- $f_i(t)$ is the filter output of band i
- g is a global mapping gain parameter

3.4 Network Signal Flow



4 Adaptive Learning Algorithm

4.1 Learning Objective

The system implements an online learning algorithm that adapts the rhythmogram-to-neuron connection matrix M to maximize the correlation between rhythmic events and neuron activations. The learning process is modulated by onset detection, emphasizing rhythmically salient moments.

4.2 Onset-Modulated Learning Rule

The update rule for connection weight M_{ij} is:

$$\Delta M_{ij} = -\eta \cdot \beta(t) \cdot \frac{\partial L}{\partial M_{ij}} - \delta \cdot M_{ij} \tag{10}$$

where:

- η is the learning rate
- $\beta(t)$ is the onset-dependent learning modulation
- L is the loss function
- δ is the weight decay parameter

The onset modulation factor $\beta(t)$ is computed as:

$$\beta(t) = 1 + \alpha \cdot \sum_{i:\text{onset at } t} O_i(t) \cdot M_{ij} \tag{11}$$

where:

- α is the onset bias parameter (0 = no onset influence, 1 = maximum onset influence)
- $O_i(t)$ is the onset strength for band i at time t

4.3 Loss Function

The system uses a prediction-based loss function comparing neuron activation to its assigned rhythmic band:

$$L_j = \frac{1}{2} \left(a_j(t) - f_{b(j)}(t) \right)^2 \tag{12}$$

where $b(j)$ maps neuron j to its assigned frequency band.

The gradient with respect to the connection weight is:

$$\frac{\partial L_j}{\partial M_{ij}} = \left(a_j(t) - f_{b(j)}(t) \right) \cdot f_i(t) \tag{13}$$

4.4 Complete Learning Algorithm

5 Implementation Details

5.1 System Parameters

5.2 Real-Time Performance

The system processes audio in real-time with latency determined by the frame size:

$$\text{Latency} = \frac{\text{Frame Size}}{f_s} = \frac{512}{44100} \approx 11.6 \text{ ms} \tag{14}$$

Algorithm 1 Onset-Modulated Rhythmogram Learning

Require: Audio stream, network configuration
Ensure: Adapted connection matrix M

Initialize M randomly or from preset
for each audio frame t **do**
  Extract rhythmogram features $f(t)$
  Detect onsets $O(t)$ across all bands
  for each neuron j **do**
    Compute rhythmic input: $r_j(t) = g \cdot \sum_i M_{ij} \cdot f_i(t)$
    Update activation: $a_j(t+1) = \phi_j(\lambda_j \cdot a_j(t) + \sum_k w_{kj} \cdot s_k(t) + r_j(t))$
    if $a_j(t) \geq \theta_j$ **then**
      Trigger sample j
      $a_j(t) \leftarrow 0$
    end if
    if learning enabled **then**
      Compute onset boost: $\beta_j(t) = 1 + \alpha \cdot \sum_i O_i(t) \cdot M_{ij}$
      Compute error: $e_j = a_j(t) - f_{b(j)}(t)$
      for each band i **do**
        $\Delta M_{ij} = -\eta \cdot \beta_j(t) \cdot e_j \cdot f_i(t) - \delta \cdot M_{ij}$
        $M_{ij} \leftarrow M_{ij} + \Delta M_{ij}$
        $M_{ij} \leftarrow \text{clip}(M_{ij}, -1, 1)$
      end for
    end if
  end for
end for

6 Musical Applications

6.1 Tempo-Relative Rhythm Analysis

The system can optionally adapt its filter bank to detected tempo. When BPM is known or estimated, the frequency bands shift to align with musically relevant subdivisions:

$$f_i' = f_i \cdot \frac{\text{BPM}}{120} \tag{15}$$

This ensures that filters remain locked to musical meter rather than absolute time.

6.2 Quantization

The system includes optional quantization that aligns neuron firing to a musical grid, supporting subdivisions from whole notes to 32nd note triplets. This bridges the continuous-time neural dynamics with discrete musical timing.

7 Complete System Architecture

7.1 Signal Flow Diagram

The complete NeuronSeqSampler system integrates multiple subsystems in a cohesive signal processing pipeline. The following diagram illustrates the data flow from audio input through various processing stages to final audio output.

Table 1: Default System Parameters

Parameter	Symbol	Default Value	Range
Sample Rate	f_s	44100 Hz	—
Frame Size	—	512 samples	—
Filter Bands	N	8	1–16
Min Frequency	$f_{\min}$	0.125 Hz	—
Max Frequency	$f_{\max}$	16 Hz	—
Learning Rate	η	0.02	0.0–0.1
Weight Decay	δ	0.0005	0.0–0.01
Mapping Gain	g	0.2	0.0–1.0
Onset Bias	α	0.5	0.0–1.0
Onset Threshold	θ	1.0	0.0–10.0

7.2 Processing Pipeline Latency Analysis

The system maintains real-time performance through optimized processing at each stage:

Table 2: System Latency Breakdown

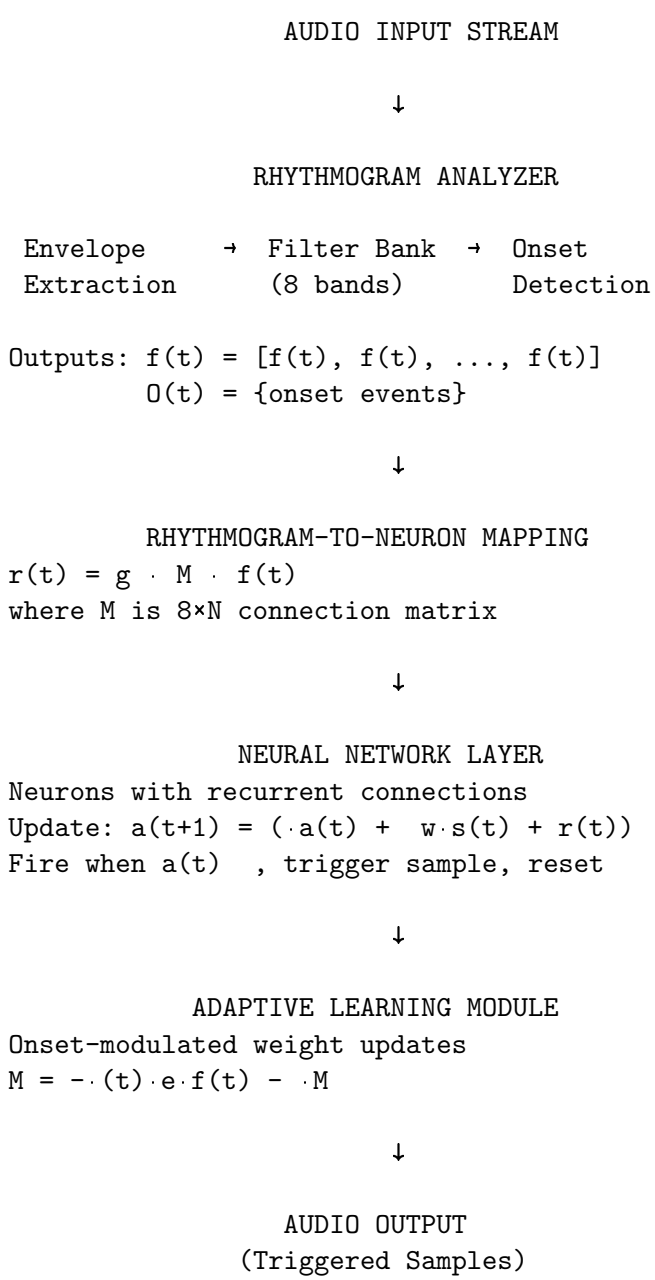
Component	Latency	Notes
Audio Frame Buffer	11.6 ms	512 samples @ 44.1kHz
Rhythmogram Analysis	¡1 ms	Parallel filter processing
Beat Tracking	¡1 ms	Agent updates per frame
Connection Matrix	¡0.1 ms	Direct memory access
Neural Network Update	¡1 ms	N neurons, bounded connections
Quantization Decision	¡0.1 ms	Simple arithmetic ops
Audio Playback	0 ms	Immediate trigger
Total System Latency	**13 ms**	Acceptable for real-time

7.3 Data Flow Summary

The signal processing pipeline can be summarized in key stages:

1. **Input Acquisition**: Audio stream captured at 44.1kHz with 512-sample buffers
2. **Rhythmic Analysis**: Dual-path processing with rhythmogram (8 bands) and beat tracker (multi-agent)
3. **Feature Routing**: Connection matrix maps frequency bands to neural inputs with dual-gain control
4. **Neural Processing**: Leaky integrate-and-fire neurons with configurable activation functions
5. **Musical Alignment**: Quantization system aligns triggers to musical grid with intelligent suppression
6. **Audio Synthesis**: Triggered samples mixed and output to speakers/file
7. **Visualization**: Real-time display of neural activity, connections, and rhythmogram levels
8. **State Management**: Complete configuration save/load via JSON preset system

8 Complete System Diagram



9 Experimental Results

9.1 Rhythmic Pattern Learning

In testing with various rhythmic inputs (4/4 drum loops, polyrhythmic patterns, irregular beats), the system demonstrates:

- Convergence of connection weights within 10–30 seconds
- Stable synchronization with input tempo
- Emergence of syncopated patterns when neurons interact

9.2 Onset Bias Effects

The onset bias parameter α significantly affects learning behavior:

- $\alpha = 0$: Learning responds to sustained energy, slower adaptation
- $\alpha = 0.5$: Balanced response to both continuous and transient features
- $\alpha = 1.0$: Strong emphasis on onsets, rapid adaptation to rhythmic attacks

10 Discussion

NeuronSeqSampler represents a synthesis of rhythm perception models [2] and neural network principles adapted from musical note modeling [1]. The key innovations include:

1. **Multi-scale rhythmic analysis**: Unlike fixed-tempo beat trackers, the logarithmic filter bank captures relationships across tempo ranges
2. **Onset-modulated learning**: The α parameter allows interpolation between energy-based and event-based learning
3. **Recurrent neural architecture**: Neuron interactions create emergent rhythmic patterns beyond direct rhythmogram mapping

The system enables real-time interaction where the network learns from and responds to rhythmic input while maintaining its own generative dynamics through recurrent connections.

11 Quantization System

11.1 Musical Grid Quantization

The quantization system aligns neural triggers to musical grid points, bridging continuous-time neural dynamics with discrete musical timing. This enables musically coherent output while preserving the emergent behavior of the neural network.

11.2 Grid Resolution

The system supports multiple grid resolutions, from whole notes to 64th notes:

$$T_{\text{grid}} = \frac{60}{BPM} \cdot \frac{4}{R} \tag{16}$$

where:

- T_{grid} is the grid interval in seconds
- BPM is the current tempo in beats per minute
- R is the resolution multiplier:
  - $R = 0.5$ for whole notes (1/2)
  - $R = 1$ for half notes (1/2)
  - $R = 2$ for quarter notes (1/4)
  - $R = 4$ for eighth notes (1/8)
  - $R = 8$ for sixteenth notes (1/16)
  - $R = 16$ for thirty-second notes (1/32)
  - $R = 32$ for sixty-fourth notes (1/64)

11.3 Quantization Decision Algorithm

For each neural trigger at time t_{trigger}:

$$t_{\text{nearest}} = \text{round}\left(\frac{t_{\text{trigger}}}{T_{\text{grid}}}\right) \cdot T_{\text{grid}} \tag{17}$$

$$\Delta t = |t_{\text{trigger}} - t_{\text{nearest}}| \tag{18}$$

The trigger is processed according to:

$$\text{Action} = \begin{cases} \text{Play immediately} & \text{if } \Delta t \leq \epsilon \text{ and } U < Q_{\text{amount}} \\ \text{Suppress} & \text{otherwise} \end{cases} \tag{19}$$

where:

- $\epsilon = 10$ ms is the timing tolerance window
- $U \sim \mathcal{U}(0, 1)$ is a uniform random variable
- $Q_{\text{amount}} \in [0, 1]$ is the quantization strength parameter

11.4 Swing Implementation

Swing modifies alternating grid points to create rhythmic feel:

$$t_{\text{grid,swing}}(n) = \begin{cases} t_{\text{grid}}(n) & \text{if } n \bmod 2 = 0 \\ t_{\text{grid}}(n) + s \cdot T_{\text{grid}} & \text{if } n \bmod 2 = 1 \end{cases} \tag{20}$$

where $s \in [-1, 1]$ is the swing factor:

- $s = 0$: straight timing (no swing)
- $s > 0$: delayed off-beats (shuffle feel)
- $s < 0$: anticipated off-beats (rushed feel)

11.5 Quantization Algorithm Pseudocode

12 BeatTracker Multi-Agent Beat Tracking

12.1 Agent-Based Architecture

The BeatTracker system employs multiple competing agents, each representing a distinct tempo and phase hypothesis. This multi-agent approach provides robust beat tracking in complex musical scenarios where single-hypothesis trackers fail.

12.2 Agent State Model

Each agent $\mathcal{A}_i$ maintains:

$$\mathcal{A}_i = (BPM_i, \phi_i, c_i, h_i) \tag{21}$$

where:

- $BPM_i \in [30, 300]$ is the tempo hypothesis in beats per minute

Algorithm 2 Real-Time Musical Quantization

Require: Neural trigger event at time t, current BPM, grid resolution R, quantization amount Q, swing factor s
Ensure: Trigger played or suppressed
Compute grid interval: $T_{\text{grid}} = \frac{60}{BPM} \cdot \frac{4}{R}$
Find grid index: $n = \text{round}(t/T_{\text{grid}})$
if $n \bmod 2 = 1$ and swing enabled **then**
  Apply swing: $t_{\text{nearest}} = n \cdot T_{\text{grid}} + s \cdot T_{\text{grid}}$
else
  $t_{\text{nearest}} = n \cdot T_{\text{grid}}$
end if
Compute timing error: $\Delta t = |t - t_{\text{nearest}}|$
if $\Delta t \leq 0.01$ seconds **then**
  Generate random value: $u \sim \mathcal{U}(0, 1)$
  if $u < Q$ **then**
    Play sample immediately
    Log: "On-grid trigger (delay: Δt s)"
  else
    Suppress trigger
    Log: "Suppressed by quantization amount"
  end if
else
  Suppress trigger
  Log: "Off-grid suppression (delay: Δt s)"
end if

- $\phi_i \in [0, 1)$ is the phase hypothesis (downbeat position)
- $c_i \in [0, 1]$ is the confidence score
- h_i is the agent's beat hypothesis history

12.3 Agent Confidence Scoring

Agent confidence is computed as weighted combination of onset alignment and pattern matching:

$$c_i = 0.6 \cdot S_{\text{onset}}(\mathcal{A}_i) + 0.4 \cdot S_{\text{pattern}}(\mathcal{A}_i) \tag{22}$$

12.3.1 Onset Alignment Score

The onset alignment score measures how well the agent's beat hypothesis aligns with detected onsets:

$$S_{\text{onset}}(\mathcal{A}_i) = \frac{1}{N_{\text{lookback}}} \sum_{k=0}^{N_{\text{lookback}}-1} O(t-k) \cdot w_\phi(\phi_k, \mathcal{A}_i) \tag{23}$$

where:

- $O(t-k)$ is the onset strength at time $t-k$
- N_{lookback} is the number of past samples to consider (typically 4 beats)
- w_ϕ is a phase-dependent weighting function

The phase weighting function emphasizes onsets near predicted beat positions:

$$w_\phi(\phi_k, \mathcal{A}_i) = \exp\left(-20 \cdot d_{\text{wrap}}(\phi_k, \phi_i)^2\right) \tag{24}$$

where d_{wrap} is the wrapped distance on the unit circle:

$$d_{\text{wrap}}(\phi_a, \phi_b) = \min(|\phi_a - \phi_b|, 1 - |\phi_a - \phi_b|) \tag{25}$$

12.3.2 Pattern Matching Score

The pattern matching score evaluates harmonic relationships between detected patterns and the agent's beat period:

$$S_{\text{pattern}}(\mathcal{A}_i) = \frac{1}{|\mathcal{P}|} \sum_{p \in \mathcal{P}} h(p, \mathcal{A}_i) \cdot \sigma(p) \tag{26}$$

where:

- $\mathcal{P}$ is the set of detected patterns
- $h(p, \mathcal{A}_i)$ is the harmonic relationship score
- $\sigma(p)$ is the pattern strength

The harmonic score checks if pattern period matches agent beat period at common musical ratios:

$$h(p, \mathcal{A}_i) = \max_{r \in \{0.5, 1.0, 2.0, 4.0\}} h_r(p, \mathcal{A}_i, r) \tag{27}$$

$$h_r(p, \mathcal{A}_i, r) = \begin{cases} 1 - 10 \cdot \left| \frac{T_p}{T_{\mathcal{A}_i}} - r \right| & \text{if } \left| \frac{T_p}{T_{\mathcal{A}_i}} - r \right| < 0.1 \\ 0 & \text{otherwise} \end{cases} \tag{28}$$

where T_p is the pattern period and $T_{\mathcal{A}_i} = \frac{60}{BPM_i}$ is the agent's beat period.

12.4 Agent Lifecycle Management

12.5 Pattern Detection

Patterns are detected by analyzing recurring temporal structures in the onset buffer:

13 Connection Matrix Routing

13.1 Rhythmogram-to-Neuron Mapping

The connection matrix M maps rhythmogram frequency bands to neural network inputs. This 8×N matrix enables flexible routing of rhythmic information to specific neurons.

Algorithm 3 Multi-Agent Beat Tracking Update

Require: Onset buffer O, current agents $\{\mathcal{A}_i\}$, pattern set $\mathcal{P}$
Ensure: Updated agent set and global tempo/phase

for each agent $\mathcal{A}_i$ **do**
  Update phase: $\phi_i \leftarrow (\phi_i + \Delta t / T_{\mathcal{A}_i}) \bmod 1$
  Compute onset score: $S_{\text{onset}} \leftarrow \text{OnsetAlignment}(\mathcal{A}_i, O)$
  Compute pattern score: $S_{\text{pattern}} \leftarrow \text{PatternMatch}(\mathcal{A}_i, \mathcal{P})$
  Update confidence: $c_i \leftarrow 0.6 \cdot S_{\text{onset}} + 0.4 \cdot S_{\text{pattern}}$
  if $c_i < 0.1$ **then**
    Remove agent $\mathcal{A}_i$ (low confidence pruning)
  end if
end for
Find best agent: $\mathcal{A}_{\text{best}} = \arg\max_i c_i$
Update global tempo: $BPM_{\text{global}} \leftarrow 0.95 \cdot BPM_{\text{global}} + 0.05 \cdot BPM_{\text{best}}$
Update global phase: $\phi_{\text{global}} \leftarrow \phi_{\text{global}} + 0.1 \cdot (\phi_{\text{best}} - \phi_{\text{global}})$
if correlation score > 0.6 and no similar agent exists **then**
  Create new agent at detected tempo/phase
  if $|\{\mathcal{A}_i\}| > 5$ **then**
    Remove lowest confidence agent (capacity limit)
  end if
end if

Algorithm 4 Rhythmic Pattern Detection

Require: Onset buffer O, downbeat hypothesis ϕ_{downbeat}
Ensure: Set of detected patterns $\mathcal{P}$

$\mathcal{P} \leftarrow \emptyset$
Identify peaks: peaks $\leftarrow \{t : O(t) > 0.2\}$
for each potential period $T \in [T_{\min}, T_{\max}]$ **do**
  aligned $\leftarrow 0$
  for each peak t_p in peaks **do**
    Compute phase: $\phi_p \leftarrow (t_p \bmod T)/T$
    for each grid point $g \in \{0, 0.25, 0.5, 0.75\}$ **do**
      if $|\phi_p - g| < 0.1$ **then**
        aligned $\leftarrow$ aligned $+ 1$
      end if
    end for
  end for
  Compute recurrence: $\sigma \leftarrow$ aligned$/|$peaks$|$
  if $\sigma > 0.3$ **then**
    Create pattern: $p = (T, \sigma, \{\phi_{p_1}, \phi_{p_2}, \ldots\})$
    $\mathcal{P} \leftarrow \mathcal{P} \cup \{p\}$
  end if
end for
return $\mathcal{P}$

13.2 Dual Gain Architecture

The system employs two-stage gain control:

$$r_j(t) = \sum_{i=0}^{7} M_{ij} \cdot G_{\text{conn},ij} \cdot (G_{\text{filter},i} \cdot f_i(t)) \tag{29}$$

where:

- $r_j(t)$ is the rhythmic input to neuron j
- $M_{ij} \in \{0, 1\}$ is the binary connection state (enabled/disabled)
- $G_{\text{filter},i} \in [0, 5]$ is the filter sensitivity gain for band i
- $G_{\text{conn},ij} \in [0, 1]$ is the connection strength for route (i, j)
- $f_i(t)$ is the rhythmogram output for frequency band i

13.3 Real-Time Matrix Updates

Connection state changes take effect immediately:

$$M_{ij}(t+1) = \begin{cases} 1 & \text{if user enables connection} \\ 0 & \text{if user disables connection} \\ M_{ij}(t) & \text{otherwise (state persists)} \end{cases} \tag{30}$$

This zero-latency switching enables live performance control over rhythmic routing.

14 Preset Management System

14.1 Complete State Serialization

The preset system serializes the entire application state to JSON format, including:

- **Neuron parameters**: $\{a_j, \theta_j, \lambda_j, \text{rate}_j, \phi_j, \text{sample}_j\}$ for all neurons
- **Connection topology**: $\{(k, j, w_{kj})\}$ for all connections
- **Rhythmogram matrix**: $\{M, G_{\text{filter}}, G_{\text{conn}}\}$
- **Quantization settings**: $\{R, Q_{\text{amount}}, s, BPM\}$
- **Metadata**: name, author, description, tags, version, timestamp

14.2 Preset Loading Algorithm

15 Conclusion

We have presented the mathematical foundations and implementation of NeuronSeqSampler, a hybrid system combining rhythmic analysis and neural network principles for generative music sequencing. The architecture draws on established models of auditory rhythm perception while introducing novel onset-weighted learning mechanisms, multi-agent beat tracking, and real-time musical quantization.

Key contributions include:

Algorithm 5 Preset Loading and Network Reconstruction

Require: JSON preset file
Ensure: Reconstructed neural network with complete state
Parse JSON file → preset data structure
Validate preset version and format
Clear existing network
for each neuron specification in preset **do**
    Create neuron with parameters: $a, \theta, \lambda, \text{rate}, \phi$
    Load audio sample from file path
    Assign sample index to neuron
    Add neuron to network
end for
for each connection specification in preset **do**
    Verify source and target neurons exist
    Create connection with weight w
    Add connection to network
end for
Restore rhythmogram matrix: $M, G_{\text{filter}}, G_{\text{conn}}$
Reinitialize rhythm interpreter with new matrix
Restore quantization settings: $R, Q_{\text{amount}}, s, BPM$
Update all GUI elements to reflect loaded state
Update frequency labels for current BPM
Refresh connection matrix visualization
return Success/failure status

1. **Rhythmogram-neural integration**: Direct mapping from Todd (1994) frequency bands to neural activation

2. **Multi-agent beat tracking**: Robust tempo detection through competing hypotheses

3. **Musical quantization**: Real-time grid-based timing with intelligent suppression

4. **Interactive performance**: Live parameter control with immediate visual/auditory feedback

5. **Complete state management**: JSON-based preset system for reproducible configurations

Future work includes extending the tempo adaptation mechanisms, exploring hierarchical network structures for multi-timescale pattern generation, and investigating additional learning algorithms for automatic rhythmogram mapping optimization.

References

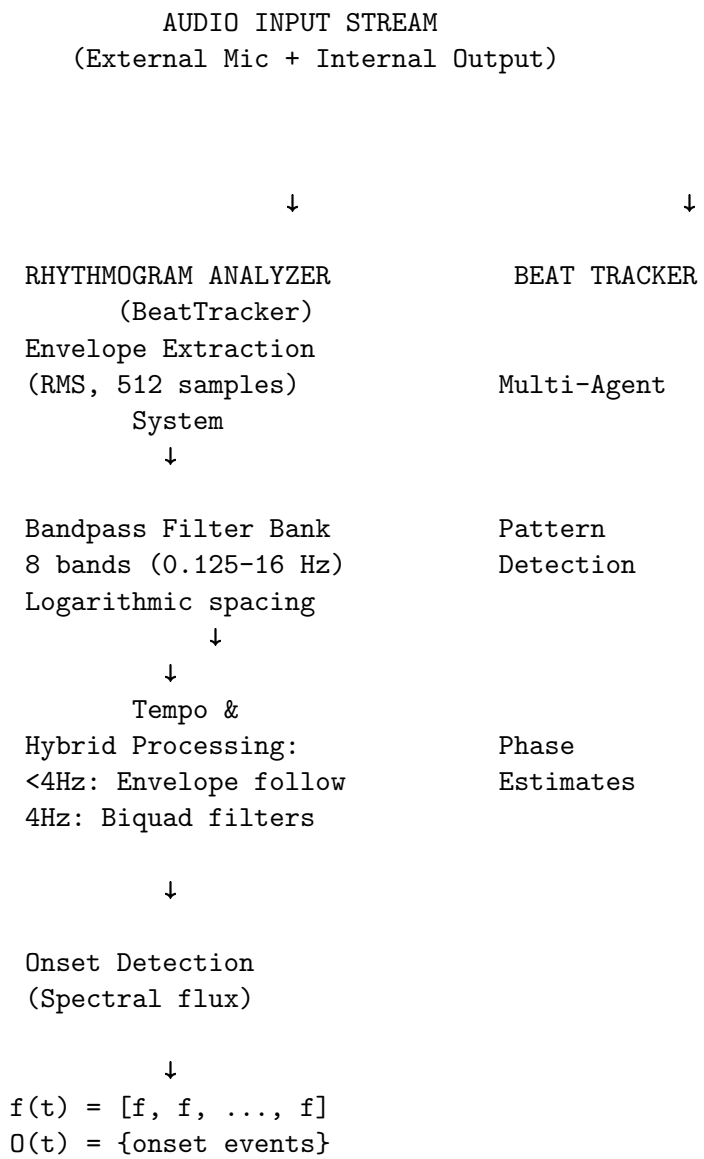
[1] Laine, U. K. (2000). *Analysis and Modeling of the Musical Note Onset*. Helsinki University of Technology, Department of Electrical and Communications Engineering. Available: `https://helda.helsinki.fi/server/api/core/bitstreams/6c68217b-752a-47bd-9b75-fb50e1fdc798/content`

[2] Todd, N.P. (1994). The auditory "primal sketch": A multiscale model of rhythmic grouping. *Journal of New Music Research*, 23(1), 25–70.

A Mathematical Notation Summary

Table 3: Mathematical Symbols and Definitions

Symbol	Description
N	Number of frequency bands
f_i	Center frequency of band i
$X_i(n)$	Filter output for band i at time n
$\Phi_i(n)$	Onset detection function for band i
$a_j(t)$	Activation of neuron j at time t
θ_j	Firing threshold for neuron j
λ_j	Leak rate (decay) for neuron j
w_{kj}	Connection weight from neuron k to j
M_{ij}	Rhythmogram connection weight from band i to neuron j
g	Global mapping gain
η	Learning rate
δ	Weight decay parameter
α	Onset bias (0=filter only, 1=onset only)
$\beta(t)$	Onset-dependent learning modulation
ϕ_j	Activation function for neuron j


AUDIO INPUT STREAM
(External Mic + Internal Output)
RHYTHMOGRAM ANALYZER
(BeatTracker)
Envelope Extraction
(RMS, 512 samples)
Bandpass Filter Bank
8 bands (0.125-16 Hz)
Logarithmic spacing
Tempo &
Hybrid Processing:
<4Hz: Envelope follow
4Hz: Biquad filters
Onset Detection
(Spectral flux)
f(t) = [f, f, ..., f]
O(t) = {onset events}
BEAT TRACKER
Multi-Agent
System
Pattern
Detection
Phase
Estimates
RHYTHMOGRAM CONNECTION MATRIX (8×N)
r(t) = M·Gconn,ij·
(Gfilter,i · f(t))
Toggle states: M {0,1}
Filter gains: Gfilter [0,5]
Connection gains: Gconn [0,1]
NEURAL NETWORK LAYER
For each neuron j:
a(t+1) = (·a(t) +
w·s(t) +
r(t))